

Falsity API Challenge

A backend system for a fact-checking platform where users submit public claims, analysts attach evidence, moderators review verdicts, and the system computes a final falsity score.

This is a very good teaching project because it naturally needs:

- authentication and authorization
- role-based access
- request validation
- middleware/hooks
- service and repository layers
- external service integration
- versioned APIs
- audit logs
- scalable schema design
- performance and indexing
- non-trivial business logic

Fastify is a strong fit because it is designed for low overhead, uses a plugin architecture with encapsulation, supports hooks across the request lifecycle, and has first-class validation/serialization patterns. Prisma is a good ORM choice for interns because it gives a schema-first model, generated type-safe client, migrations, and production-focused best practices.

1) What interns will build

They will build a backend for a platform where:

1. A **citizen/user** submits a claim
Example: "Company X reduced 80% pollution in 3 months."
2. The claim enters a review pipeline
3. An **analyst** collects evidence from internal notes and external sources
4. A **moderator/reviewer** approves or rejects the analysis
5. The system assigns:
 - verdict: **TRUE | MISLEADING | PARTLY_FALSE | FALSE | UNVERIFIABLE**
 - confidence score
 - evidence weight
 - audit trail of all actions
6. Public consumers can query published verdicts

This gives interns exposure to both simple CRUD and advanced workflow design.

2) Core learning goals

By the end, interns should understand:

- how a request flows from
route → **versioned module** → **controller** → **service** → **repository** → **database**
- where validation should happen
- when to use hooks/middleware vs service logic
- why repositories exist
- why internal services and external services should be separated
- how auth and authorization differ
- how schema design affects scalability
- why ORMs are useful and where raw SQL may still be needed
- how to build consistent error handling and response shapes
- how to design for maintainability, not just “it works”

Fastify’s docs explicitly emphasize plugins, hook lifecycle, and validation/serialization patterns, which makes it ideal for teaching these boundaries cleanly.

3) Recommended tech stack

Main stack

- **Node.js + TypeScript**
- **Fastify**
- **Prisma ORM**
- **PostgreSQL**
- **JWT auth**
- **Redis** for caching/rate limit/session support later
- **Pino logger** via Fastify logging
- **Zod or JSON Schema** for request validation
- **Docker** for local setup
- **Vitest/Jest** for tests

Why this stack

Why Fastify

Use Fastify to teach:

- route registration
- plugin encapsulation
- hooks
- schema validation
- serialization
- decorators
- modular architecture

Fastify plugins create encapsulated scopes by default, which is a great real-world concept for interns because it prevents uncontrolled global coupling.

Why Prisma

Use Prisma to teach:

- DB schema modeling
- migrations
- relations
- indexes
- type-safe DB access
- transactions
- developer productivity

Prisma's official docs describe it as a schema-based, type-safe database toolkit with migrations and best-practice guidance for production schema design and query optimization.

4) The business domain

Main modules

1. **Auth**
2. **Users**
3. **Claims**
4. **Evidence**
5. **Investigations**
6. **Verdicts**
7. **Comments / Review notes**
8. **Audit logs**

- 9. Notifications
- 10. External source ingestion

User roles

- USER
- ANALYST
- REVIEWER
- ADMIN

Core lifecycle

DRAFT -> SUBMITTED -> UNDER_REVIEW -> NEEDS_MORE_EVIDENCE ->
READY_FOR_VERDICT -> PUBLISHED -> ARCHIVED

This teaches state machines and workflow constraints.

5) The logical challenge part

This is the most important piece.

Challenge: Verdict Engine

Interns must design a **verdict computation engine**.

Each claim can have many evidence records:

- source type: government, news, research paper, blog, social post, internal report
- credibility score
- freshness score
- contradiction/support score
- reviewer confidence
- relevance score

The system should produce:

- final falsity score: 0-100
- verdict label
- confidence band

Why this is hard

Because not every evidence item should count equally.

Example rules

- Government or peer-reviewed sources carry higher base weight
- Old evidence loses freshness weight
- Duplicate sources should not inflate score
- Direct evidence weighs more than indirect commentary
- Conflicting evidence should reduce confidence
- A claim with too little strong evidence becomes UNVERIFIABLE

Example formula idea

Not exact code, just concept:

```
final_score = weighted_support - weighted_contradiction ±  
freshness_adjustment ± source_quality_adjustment
```

Then map:

- 0-20 → TRUE
- 21-40 → MISLEADING
- 41-60 → PARTLY_FALSE
- 61-80 → FALSE
- insufficient high-quality evidence → UNVERIFIABLE

This is where they learn

- domain modeling
- business rules
- pure functions
- testability
- idempotency
- edge cases
- separating domain logic from transport logic

This is the part that forces them to think like backend engineers, not just API coders.

6) Step-up learning roadmap

Design the project in 8 phases.

Phase 1 — Foundation

Teach:

- Fastify server setup
- health route
- env config
- API versioning
- basic folder structure
- logging

Deliverables:

- `GET /health`
- `GET /api/v1/ping`
- config loader
- global error handler
- response formatter

Phase 2 — Users and auth basics

Teach:

- signup/login
- password hashing
- JWT
- auth middleware/hook
- role basics

Deliverables:

- register
- login
- get profile
- protect private routes

Phase 3 — Claims CRUD

Teach:

- controllers/services/repositories
- validation
- pagination
- filtering
- soft delete
- ownership checks

Deliverables:

- create claim

- list claims
- get claim by id
- update own draft claim
- submit claim

Phase 4 — Evidence module

Teach:

- one-to-many relations
- file metadata or link metadata
- external service integration
- transactions

Deliverables:

- add evidence to claim
- classify source
- score source
- prevent duplicate evidence

Phase 5 — Investigation workflow

Teach:

- state transitions
- role-based authorization
- internal comments
- reviewer notes
- audit logs

Deliverables:

- assign analyst
- move to under review
- request more evidence
- mark ready for verdict

Phase 6 — Verdict engine

Teach:

- domain service
- pure business logic
- test coverage
- aggregation
- conflict handling

Deliverables:

- compute score
- generate verdict
- publish verdict

Phase 7 — Performance and scale

Teach:

- indexes
- query optimization
- response serialization
- caching
- N+1 awareness
- DB transactions

Deliverables:

- leaderboard/trending claims
- optimized public claim listing
- caching hot endpoints

Phase 8 — Production readiness

Teach:

- rate limiting
- structured logging
- security headers
- monitoring hooks
- graceful shutdown
- CI/CD basics

Deliverables:

- production checklist
- error tracking integration
- API docs
- tests

7) Industry-style folder structure

Here is a strong, beginner-friendly but scalable structure:

src/

app.ts

server.ts

config/

env.ts

logger.ts

docs.ts

constants.ts

plugins/

prisma.ts

jwt.ts

redis.ts

swagger.ts

common/

errors/

app-error.ts

auth-error.ts

validation-error.ts

not-found-error.ts

conflict-error.ts

responses/

api-response.ts

paginated-response.ts

hooks/

request-id.hook.ts

auth-context.hook.ts

response-time.hook.ts

utils/

pagination.ts

slug.ts

date.ts

hashing.ts

types/

auth.ts

request.ts

modules/

auth/

auth.routes.ts

auth.controller.ts

auth.service.ts

auth.repository.ts

auth.schema.ts

auth.types.ts

users/

- users.routes.ts
- users.controller.ts
- users.service.ts
- users.repository.ts
- users.schema.ts

claims/

- claims.routes.ts
- claims.controller.ts
- claims.service.ts
- claims.repository.ts
- claims.schema.ts
- claims.mapper.ts

evidence/

- evidence.routes.ts
- evidence.controller.ts
- evidence.service.ts
- evidence.repository.ts
- evidence.schema.ts
- evidence-scoring.service.ts

investigations/

- investigations.routes.ts
- investigations.controller.ts
- investigations.service.ts
- investigations.repository.ts
- investigations-policy.service.ts

verdicts/

- verdicts.routes.ts
- verdicts.controller.ts
- verdicts.service.ts
- verdicts.repository.ts
- verdict-engine.service.ts
- verdict.schema.ts

audit/

- audit.service.ts
- audit.repository.ts

external-sources/

- external-sources.service.ts
- source-normalizer.ts
- source-http.client.ts

routes/

index.ts
v1.ts

prisma/
 schema.prisma
 migrations/
 seed.ts

tests/
 unit/
 integration/
 e2e/

Why this structure works

modules/

Feature-based grouping is easier to scale than grouping by file type across the whole app.

controller

Handles HTTP concerns only:

- request input
- calling service
- sending response

service

Contains business logic:

- validations that depend on DB state
- orchestration
- transactions
- workflow rules

repository

Handles persistence:

- Prisma queries
- filtering
- joins/include/select
- DB-specific operations

external-sources/

Anything calling outside APIs belongs here, not in repositories.

common/

Reusable cross-cutting logic:

- errors
 - response wrappers
 - shared hooks
 - utility helpers
-

8) Data flow you want interns to understand

Use this exact mental model:

Request flow

Route

Defines endpoint, method, schema, auth requirement

↓

Version module (/api/v1)

Groups versioned routes

↓

Controller

Reads request params/body/query/user context

↓

Service

Applies business rules and orchestration

↓

Repository

Reads/writes database

↓

Database

↓

Back upward with structured result

Example

POST /api/v1/claims/:id/evidence

- route validates body schema
- auth hook ensures user is logged in
- controller extracts `claimId`, `sourceUrl`, `sourceType`
- service checks claim state and role
- external service fetches source metadata if needed
- repository stores evidence
- audit service records action
- response class returns uniform output

That separation is what makes the code understandable and scalable.

9) What is the use of repositories?

A lot of beginners ask this.

Repository purpose

A repository is the **database access boundary**.

It helps because:

- controllers do not know SQL/Prisma
- services do not become cluttered with query details
- database logic is reusable
- testing becomes easier
- switching query patterns is easier

Good example

Bad:

- service directly writes huge Prisma query everywhere

Better:

- service says `claimsRepository.findClaimWithEvidence(id)`
- repository contains the actual DB query

Rule

Repository should handle:

- queries
- includes/selects
- pagination query building
- transactions helper participation

Repository should **not** handle:

- HTTP request logic
 - response formatting
 - complex domain policy decisions
-

10) Internal service vs external service

Internal service

Business logic fully inside your system.

Examples:

- claim submission rules
- verdict calculation
- state transition validation
- permission checks

External service

Calls outside your app.

Examples:

- fetch metadata from source URL
- verify email
- send notifications
- call anti-spam or AI classification API

Why separate them

Because external services are:

- slower
- unreliable
- retry-prone

- timeout-prone
- dependent on network

So keep them isolated.

Example:

- `EvidenceService` decides whether metadata is needed
- `SourceHttpClient` fetches metadata from external source
- repository stores results

That keeps business logic clean.

11) Good database practices to teach

Use PostgreSQL

For this project, PostgreSQL is better than MongoDB because interns can learn:

- relational modeling
- normalization
- constraints
- indexes
- transactions
- foreign keys

Good DB practices

1. Use UUIDs for public-facing ids

Safer than incremental ids for exposed APIs.

2. Add timestamps everywhere

- `createdAt`
- `updatedAt`

For workflow tables also add:

- `submittedAt`
- `publishedAt`
- `reviewedAt`

3. Use soft delete carefully

Prefer `archivedAt` or `deletedAt` for critical entities like claims, not hard delete.

4. Add indexes on frequent filters

Examples:

- `claims(status, createdAt)`
- `claims(submittedById)`
- `evidence(claimId)`
- `verdicts(claimId)`
- `audit_logs(entityType, entityId)`

5. Use foreign keys

They protect integrity.

6. Keep enums for stable business states

Examples:

- `role`
- `claim status`
- `verdict type`

7. Avoid storing too much derived data

Store calculated data only when:

- it is expensive to recompute
- it is needed frequently
- you know how to keep it consistent

8. Use transactions for workflow operations

Example:

“publish verdict” may need:

- update investigation
- create verdict
- create audit log
- notify subscribers

That should be one transaction.

Prisma's docs explicitly highlight schema design, query optimization, and production-ready patterns as best practices, which supports teaching interns these decisions early.

12) How to design a good schema

Below is a strong schema design outline.

Main tables

users

- id
- name
- email
- passwordHash
- role
- isActive
- createdAt
- updatedAt

claims

- id
- title
- statement
- category
- submittedById
- status
- currentAnalystId
- currentReviewerId
- publicSlug
- submittedAt
- publishedAt
- createdAt
- updatedAt
- archivedAt

evidence

- id
- claimId
- type
- sourceUrl
- sourceTitle
- sourcePublisher
- publicationDate

- stance (SUPPORTS | CONTRADICTS | MIXED | CONTEXTUAL)
- credibilityScore
- freshnessScore
- relevanceScore
- addedById
- createdAt

investigations

- id
- claimId
- assignedAnalystId
- summary
- internalNotes
- reviewStatus
- createdAt
- updatedAt

verdicts

- id
- claimId
- verdict
- falsityScore
- confidenceScore
- reasoning
- publishedById
- createdAt
- publishedAt

comments

- id
- claimId
- authorId
- visibility (INTERNAL | REVIEWER_ONLY | PUBLIC)
- content
- createdAt

audit_logs

- id
- entityType
- entityId
- action
- actorId
- metadataJson

- createdAt

Schema design rules to teach

- one table = one responsibility
 - avoid giant JSON fields for core relational data
 - use JSON only for flexible metadata, not primary design
 - define relation names clearly
 - avoid nullable chaos unless business meaning is clear
 - think about query patterns before final schema
-

13) What is the use of an ORM?

ORM purpose

An ORM lets developers work with the database using code objects/models rather than writing raw SQL for everything.

Why use ORM here

For interns, ORM helps them learn:

- schema modeling
- relationships
- migrations
- CRUD
- type-safe queries
- transactions

Prisma especially helps because it generates a typed client from the schema and improves confidence while querying.

But also teach this important truth

ORM is not magic.

They should also learn:

- indexes still matter
- query shape still matters
- N+1 problems still exist
- sometimes raw SQL is better for analytics/heavy aggregation

Best mindset:

- use ORM for most business queries
 - use raw SQL selectively for very complex performance-critical paths
-

14) Auth flow: how it works

Authentication

Authentication answers:

Who are you?

Example:

- user logs in with email + password
- backend verifies password hash
- backend issues JWT access token

Authorization

Authorization answers:

What are you allowed to do?

Example:

- analyst can add evidence
- reviewer can publish verdict
- user can edit only their own draft claim
- admin can manage all users

Recommended auth flow

1. Register
2. Hash password
3. Store user
4. Login
5. Verify credentials
6. Sign JWT
7. Send token
8. Client sends token in `Authorization: Bearer <token>`
9. auth hook verifies token

10. request gets `user` context attached
11. controller/service checks permissions

Fastify's JWT ecosystem and hook system make this pattern natural, and Fastify's lifecycle docs are useful for teaching where verification hooks belong.

Teaching note

Make interns clearly understand:

- **authentication** = identity
- **authorization** = permissions

Most beginners mix these up.

15) How middleware/hooks should be used

In Fastify, hooks are part of the request/response lifecycle. That makes them ideal for cross-cutting concerns, but not for core business logic.

Good use cases for hooks

- request id creation
- logging
- JWT verification
- rate limit checks
- response timing
- serialization tweaks
- attaching user context

Bad use cases for hooks

- calculating verdicts
- moving workflow states
- performing complex DB-heavy domain rules

Those belong in services.

Intern rule

If logic is about **all routes / cross-cutting behavior**, use a hook.
If logic is about **business domain decisions**, use a service.

16) Request validation

Validation should happen as early as possible.

Teach 3 layers of validation

Layer 1 — request schema validation

Checks:

- required fields
- types
- min/max lengths
- enum values
- query params

Fastify has built-in validation/serialization patterns, and consistent schema use is one of its biggest strengths.

Layer 2 — business validation

Checks:

- claim must be in DRAFT before submit
- analyst cannot publish verdict
- evidence source URL cannot be duplicated for same claim

Layer 3 — database validation

Checks:

- unique constraints
- foreign keys
- not-null constraints

Teach interns that request validation is not enough by itself.

17) Uniform response classes

This is very important for industry-style APIs.

Why uniform responses?

Because frontend teams and API consumers want predictability.

Example response shapes

Success

```
{
  "success": true,
  "message": "Claim created successfully",
  "data": {
    "id": "uuid",
    "title": "..."
  },
  "meta": null
}
```

Paginated

```
{
  "success": true,
  "message": "Claims fetched successfully",
  "data": [...],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 240,
    "totalPages": 12
  }
}
```

Error

```
{
  "success": false,
  "message": "Validation failed",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [...]
  }
}
```

Teach interns

- controllers should not invent random response shapes
 - use one response helper/wrapper everywhere
 - errors should also be standardized
-

18) Common error handling throughout the code

Fastify supports custom error handling, and consistent async patterns matter for stable behavior. Its docs note that mixed patterns and incorrect validator behavior can create handling problems.

Error handling pattern to teach

Create base error class

```
class AppError extends Error {  
  statusCode: number;  
  code: string;  
  details?: unknown;  
}
```

Derived errors

- `ValidationError`
- `UnauthorizedError`
- `ForbiddenError`
- `NotFoundError`
- `ConflictError`
- `ExternalServiceError`

Global error handler

Use one Fastify error handler that:

- logs the error
- hides sensitive details in production
- returns uniform error response
- maps known error classes to status codes

Golden rule

- throw errors in services
 - handle them globally
 - do not wrap everything in random try/catch blocks unless needed
-

19) API versioning

Teach interns to version from the start.

Why versioning matters

Because APIs evolve.

Example:

- `/api/v1/claims`
- `/api/v2/claims`

Maybe in v2 you:

- add better filters
- change evidence payload
- split public/private fields

How to teach it

Create:

- `routes/v1.ts`
- register each module under `/api/v1`

This helps them understand long-term API evolution.

20) A clean example of request flow

Use this as a teaching example:

Endpoint

`POST /api/v1/claims/:claimId/verdict/publish`

Route

- requires JWT
- requires role `REVIEWER`
- validates params/body

Controller

- extracts `claimId`
- gets authenticated user
- passes command to service

Verdict service

- loads claim
- checks state is `READY_FOR_VERDICT`
- loads all evidence
- calls verdict engine
- creates verdict in transaction
- marks claim as `PUBLISHED`
- writes audit log
- returns result

Repository

- `claimsRepository.findByIdWithEvidence`
- `verdictRepository.create`
- `auditRepository.create`

Response

Uniform success response

This single endpoint teaches almost the entire backend architecture.

21) Suggested API endpoints

Auth

- `POST /api/v1/auth/register`

- `POST /api/v1/auth/login`
- `GET /api/v1/auth/me`

Users

- `GET /api/v1/users/me`
- `GET /api/v1/users/:id`
- `PATCH /api/v1/users/:id/role` (admin)

Claims

- `POST /api/v1/claims`
- `GET /api/v1/claims`
- `GET /api/v1/claims/:id`
- `PATCH /api/v1/claims/:id`
- `POST /api/v1/claims/:id/submit`
- `POST /api/v1/claims/:id/archive`

Evidence

- `POST /api/v1/claims/:id/evidence`
- `GET /api/v1/claims/:id/evidence`
- `PATCH /api/v1/evidence/:id`
- `DELETE /api/v1/evidence/:id`

Investigation

- `POST /api/v1/claims/:id/assign-analyst`
- `POST /api/v1/claims/:id/request-more-evidence`
- `POST /api/v1/claims/:id/ready-for-verdict`

Verdicts

- `POST /api/v1/claims/:id/compute-verdict`
- `POST /api/v1/claims/:id/publish-verdict`
- `GET /api/v1/verdicts/:id`

Audit

- `GET /api/v1/claims/:id/audit-logs`

22) How to teach scalability and optimization

Since they are beginners, teach optimization in levels.

Level 1

- use pagination
- never return all rows
- select only needed fields
- add indexes

Level 2

- avoid N+1 queries
- use joins/includes wisely
- cache public hot endpoints
- use background jobs later for heavy work

Level 3

- transactional consistency
- idempotent write endpoints
- retry-safe external integrations
- query profiling

Good challenge examples

- public verdict listing with filters and sorting
 - trending false claims endpoint
 - dashboard stats endpoint with optimized aggregation
-

23) Mini business rules to include in the assignment

These make the project realistic.

1. A normal user can only edit a claim while status is **DRAFT**
2. Once submitted, only analyst/reviewer/admin can modify workflow fields
3. Evidence cannot be duplicated by same normalized URL for same claim
4. A verdict cannot be published unless:
 - at least 3 evidence items exist
 - at least 1 high-credibility source exists
 - investigation summary is present
5. Reviewer cannot approve their own analyst work unless admin override is enabled
6. Every state transition must create an audit log entry
7. Public verdicts expose fewer fields than internal investigation endpoints

These rules naturally teach validation, authorization, schema design, and domain services.

24) Recommended assignment format for interns

Part A — Basic backend setup

Goal: build project skeleton and health/version routes

Part B — Auth and roles

Goal: implement secure login and protected routes

Part C — Claims workflow

Goal: create and manage claims

Part D — Evidence and investigation

Goal: build review pipeline

Part E — Verdict engine

Goal: solve the logical challenge

Part F — Production hardening

Goal: logging, tests, caching, error handling, docs

25) What to evaluate in intern submissions

Architecture

- good separation of concerns
- consistent folder structure
- proper use of services/repositories

Code quality

- naming clarity
- no duplicated logic
- consistent responses/errors

Security

- password hashing
- protected routes
- role checks
- input validation

Database quality

- good schema
- useful indexes
- relation design
- migrations

Business logic

- correct state transitions
- verdict engine correctness
- edge case handling

Performance

- pagination
- query optimization

- no unnecessary data loading

Testing

- unit tests for verdict engine
 - integration tests for protected endpoints
 - e2e tests for core flow
-

26) Best way to explain “where to apply which concept”

Here is a simple mapping table in words.

Use route layer for

- endpoint declaration
- schema registration
- auth pre-handlers/hook wiring

Use controller for

- extracting request data
- calling service
- returning formatted response

Use service for

- business rules
- orchestration
- transactions
- calling repositories and internal/external services

Use repository for

- DB reads/writes only

Use hook/middleware for

- logging

- auth token parsing
- request ID
- rate limiting
- cross-cutting concerns

Use external service layer for

- third-party APIs
- email/SMS
- metadata enrichment

Use common utils for

- shared pure helpers
- pagination calculations
- hashing/date formatting

This is the separation interns must internalize.

27) Final project statement you can give interns

You can directly paste this into your internship doc:

Build a scalable backend API called **Falsity API Challenge** using **Fastify**, **TypeScript**, **Prisma**, and **PostgreSQL**. The system allows users to submit claims, analysts to attach evidence and investigate them, reviewers to publish verdicts, and public consumers to query published outcomes.

Your goal is not only to make endpoints work, but to design the backend in a clean industry-style architecture using:

- API versioning
- modular folder structure
- controllers, services, repositories
- auth and role-based authorization
- request validation
- uniform success/error responses
- global error handling
- audit logs
- optimized schema design and indexing
- scalable data flow from route to database

The most important part is the **Verdict Engine**, where you must design business logic to compute a falsity score from multiple evidence sources with different quality, freshness, and credibility levels.

28) My recommendation for the teaching order

For beginners, do **not** start with the full project at once.

Teach in this order:

1. Fastify basics
2. route/controller/service flow
3. Prisma schema + CRUD
4. auth
5. authorization
6. hooks/middleware
7. repositories
8. workflow states
9. external services
10. verdict engine
11. caching/performance
12. testing

That order makes the flow much clearer.

29) The single most important teaching principle

Tell interns this:

A good backend is not a collection of endpoints. It is a well-structured system where each layer has one job.

That is the core lesson of this project.

If you want, I can turn this into a **full internship-ready project document** with:

- folder-wise explanation
- Prisma schema draft

- endpoint list
- step-by-step weekly plan
- evaluation rubric
- sample controller/service/repository code structure